

# UT-04: Estructuras de control

¿Qué vamos a ver?

- Sentencias condicionales
  - Condicional simple: IF
  - Condicional doble: IF – ELSE
  - Anidación de sentencias condicionales
  - Condicional múltiple SWITCH-CASE
- Sentencias iterativas
  - WHILE
  - DO-WHILE
  - FOR
- Sentencias de salida de un bucle
  - Break
  - Continue
- Prueba, depuración y documentación de la aplicación

## 1. SENTENCIAS CONDICIONALES

Las sentencias condicionales determinan si se ejecuta una instrucción, o un conjunto de instrucciones, en función de si se cumple, o no se cumple, una condición.

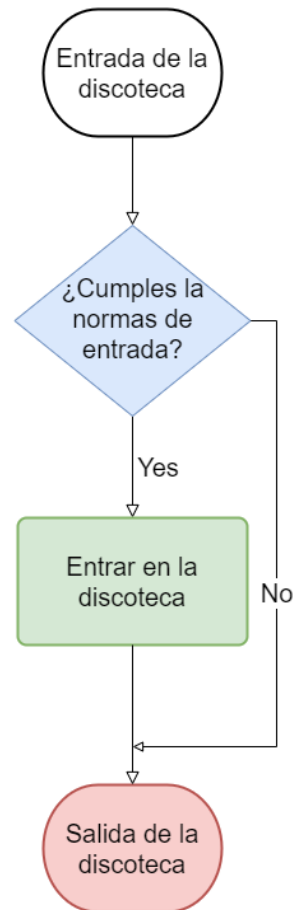
Existen con tres tipos de sentencias condicionales, en función del número de posibles valores que pueda tomar la condición:

- Simples
- Dobles
- Múltiple

### 1.1. Sentencia condicional simple (if)

Las sentencias condicionales simples son las que ejecutan una instrucción o un conjunto de instrucciones únicamente si se cumple una determinada condición.





En el lenguaje de programación Java, la sentencia condicional simple está representada por la instrucción "if".

```
if (condición) {  
    instrucción o bloque de instrucciones a ejecutar  
}
```

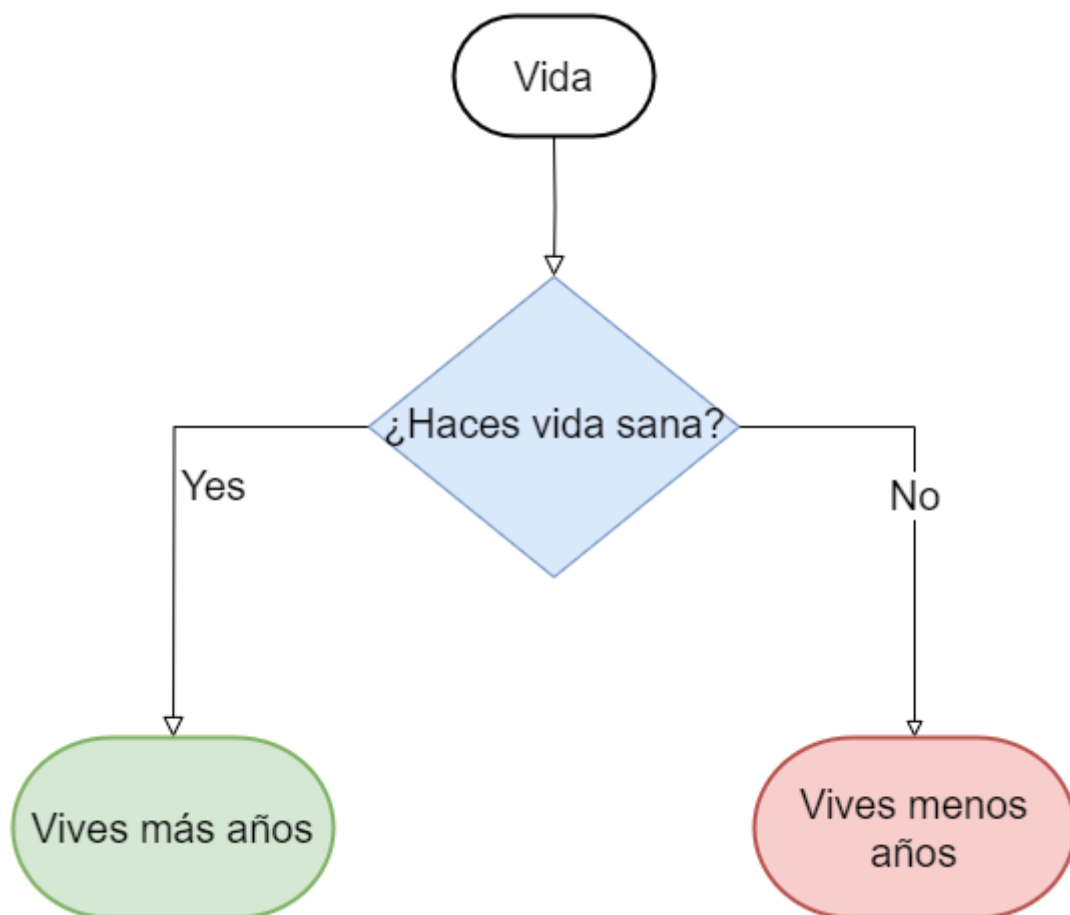
```
if (numDiaSemana == 6) {  
    System.out.println("¡Es sábado!");  
}
```

Cuando sólo vamos a ejecutar una única instrucción dentro del if, podemos prescindir de los corchetes.

```
if (numDiaSemana == 6)  
    System.out.println("¡Es sábado!");
```

### 1.1. Sentencia condicional simple (if-else)

Las sentencias condicionales dobles son las que ejecutan una instrucción o un bloque de instrucciones si se cumple una determinada condición, y si no se cumple se ejecuta otra instrucción u otro bloque de instrucciones.



Vamos a verlo ejemplificado en un vídeo...

Para realizar esta acción en Java vamos a añadir un complemento a la instrucción if, el else. De tal forma, que si se cumple la condición se ejecutará la parte dentro del if y si no se cumple se ejecutará la parte del else.

```
if (condición) {
```

```
    bloque de instrucciones a ejecutar si se cumple la condición
```

```
}
```

```
else {
```

```
    bloque de instrucciones a ejecutar si no se cumple la condición
```

```
}
```

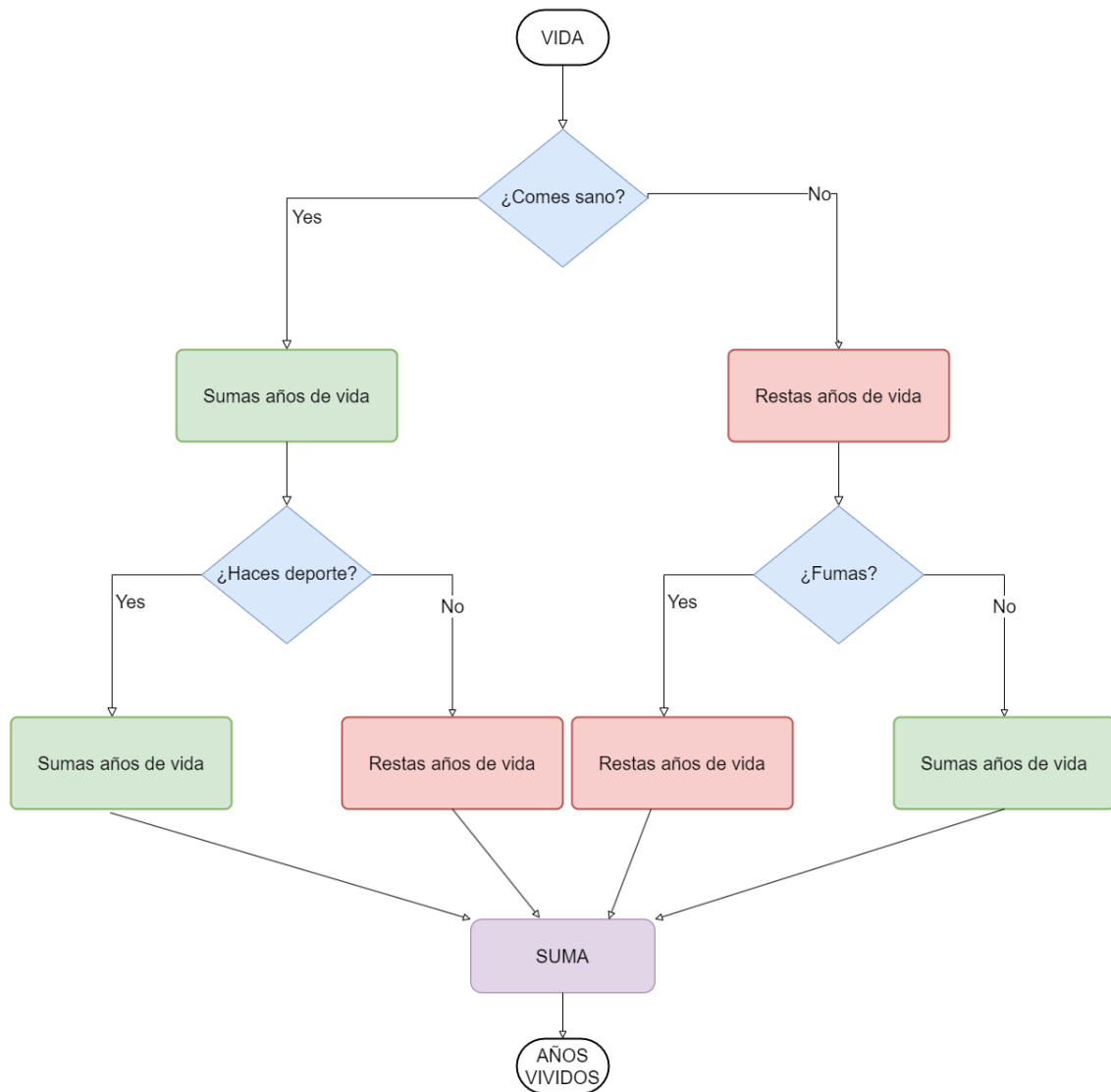
```
if (numero % 2 == 0){  
    System.out.println(numero + " es par");  
}  
else {  
    System.out.println(numero + " es impar");  
}
```

Cuando sólo vamos a ejecutar una única instrucción dentro del if, podemos prescindir de los corchetes.

```
if (numero % 2 == 0)  
    System.out.println(numero + " es par");  
else  
    System.out.println(numero + " es impar");
```

## 1.2. Anidamiento de sentencias condicionales

En ocasiones necesitaremos utilizar sentencias condicionales dentro de otras sentencias condicionales.



Para representar este anidamiento en Java solo tendremos que introducir una instrucción if dentro de otra instrucción if o else:

```

if (num1!=num2){
    if (num1 > num2){
        System.out.println(num1 + " es mayor que " + num2);
    }
    else {
        System.out.println(num1 + " es mayor que " + num2);
    }
} else {
    System.out.println("Los dos números son iguales");
}

```

```
if (num1==num2){  
    System.out.println("Los dos números son iguales");  
} else {  
    if (num1 > num2){  
        System.out.println(num1 + " es mayor que " + num2);  
    }  
    else {  
        System.out.println(num1 + " es mayor que " + num2);  
    }  
}
```

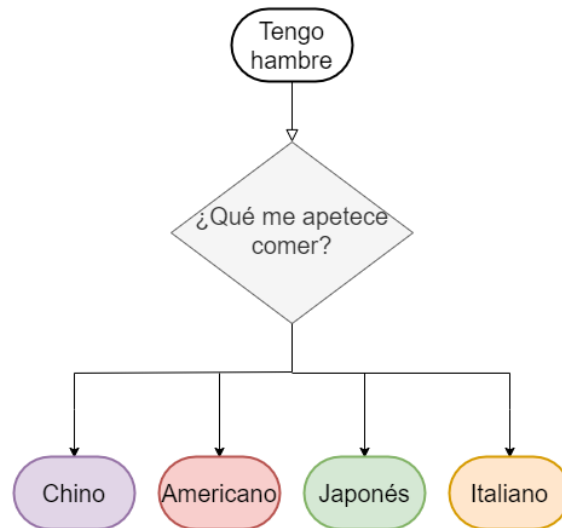
En el caso de incluir dentro de un else un nuevo if, podremos hacerlo de la siguiente manera:

```
if (num1==num2){  
    System.out.println("Los dos números son iguales");  
} else if (num1 > num2){  
    System.out.println(num1 + " es mayor que " + num2);  
}  
else {  
    System.out.println(num1 + " es mayor que " + num2);  
}
```



### 1.3. Condicional múltiple SWITCH-CASE

En determinadas ocasiones tendremos varias opciones, no solo dos, como resultado de una expresión o condición.



En estos casos podríamos utilizar anidamientos de expresiones condicionales, pero el código quedaría bastante engorroso, por ejemplo:

```
System.out.print("Dame un número de 0 a 3: ");
num = scan.nextInt();

if (num==0){
    System.out.println("El número es 0");
}
else {
    if (num==0) {
        System.out.println("El número es 1");
    }
    else {
        if (num==0) {
            System.out.println("El número es 2");
        }
        else {
            if (num==0) {
                System.out.println("El número es 3");
            }
        }
    }
}
}
```



Para evitar esta situación, contamos una sentencia condicional múltiple en Java denominada SWITCH-CASE, de tal forma que la traducción del anterior código a este sería el siguiente:

```
switch (num){  
    case 0:  
        System.out.println("El número es 0");  
        break;  
    case 1:  
        System.out.println("El número es 1");  
        break;  
    case 2:  
        System.out.println("El número es 2");  
        break;  
    case 3:  
        System.out.println("El número es 3");  
        break;  
}
```

Esto nos permitirá tener un código mucho más claro.

La instrucción SWITCH-CASE consiste en proporcionar la expresión que queremos evaluar en el paréntesis después de la palabra reservada “switch” y, a continuación, le pondremos por cada opción la palabra reservada “case” seguido del valor al que corresponde dicha opción. Seguiremos con un signo de dos puntos (“:”) y escribiremos el bloque de código que necesitemos ejecutar cuando obtengamos dicha opción. Finalizaremos dicho bloque con la palabra reservada “break”.

Además, esta instrucción nos permite contemplar la opción de que no se cumpla ninguna de las que hemos definido mediante la palabra reservada “default”, la cual utilizaremos como una opción más y donde introduciremos el bloque de código que queremos que se ejecute si no se cumple ninguna de las opciones anteriores.

```
switch (num){  
    case 0:  
        System.out.println("El número es 0");  
        break;  
    case 1:  
        System.out.println("El número es 1");  
        break;  
    case 2:  
        System.out.println("El número es 2");  
        break;  
    case 3:  
        System.out.println("El número es 3");  
        break;  
    default:  
        System.out.println("El número proporcionado no es correcto");  
        break;  
}
```

Veamos un pequeño vídeo para entender mejor cómo funciona la sentencia switch case.

La sentencia switch case nos permite ejecutar un bloque de instrucciones para varias opciones. Para hacer esto, tenemos las dos siguientes opciones:

```
switch (piso){  
    case 1, 2, 3:  
        System.out.println("El piso es bajo");  
        break;  
    case 4, 5, 6:  
        System.out.println("El piso es medio");  
        break;  
    case 7, 8, 9:  
        System.out.println("El piso es alto");  
        break;  
}
```

```
switch (piso){
    case 1:
    case 2:
    case 3:
        System.out.println("El piso es bajo");
        break;
    case 4:
    case 5:
    case 6:
        System.out.println("El piso es medio");
        break;
    case 7:
    case 8:
    case 9:
        System.out.println("El piso es alto");
        break;
}
```

También podemos hacer que una opción ejecute varios bloques de instrucciones seguido retirando la palabra reservada “break” entre ellas.

```
switch (opcion){
    case 0:
        System.out.println("Hola ¿qué tal?");
    case 1:
        System.out.println("¿Cómo estás?");
        break;
    case 2:
        System.out.println("Yo estoy bien");
    case 3:
        System.out.println("¿Y tú?");
        break;
    default:
        System.out.println("Nada que decir");
        break;
}
```

En este ejemplo las salidas, según el valor de opción, serían:

- 0 → “Hola ¿Qué tal?”  
“¿Cómo estás?”
- 1 → “¿Cómo estás?”
- 2 → “Yo estoy bien”  
“¿Y tú?”

- 3 → “¿Y tú?”
- Default → “Nada que decir”

## 2. SENTENCIAS ITERATIVAS

Las sentencias iterativas nos permitirán repetir bloques de instrucciones mientras, o hasta, que se cumpla una condición un número determinado o indeterminado de veces.

Son instrucciones con una gran potencia y que nos permitirán proporcionar gran flexibilidad a nuestros programas combinadas con las sentencias condicionales, pero también debemos tener cuidado a la hora de definirlos, puesto que podemos no contemplar todas las posibles condiciones y provocar que entremos un bucle infinito. Esto provocará que nuestro programa no salga nunca del bucle y por tanto no llegue a finalizar.

Mira el vídeo para entender a qué nos referimos...

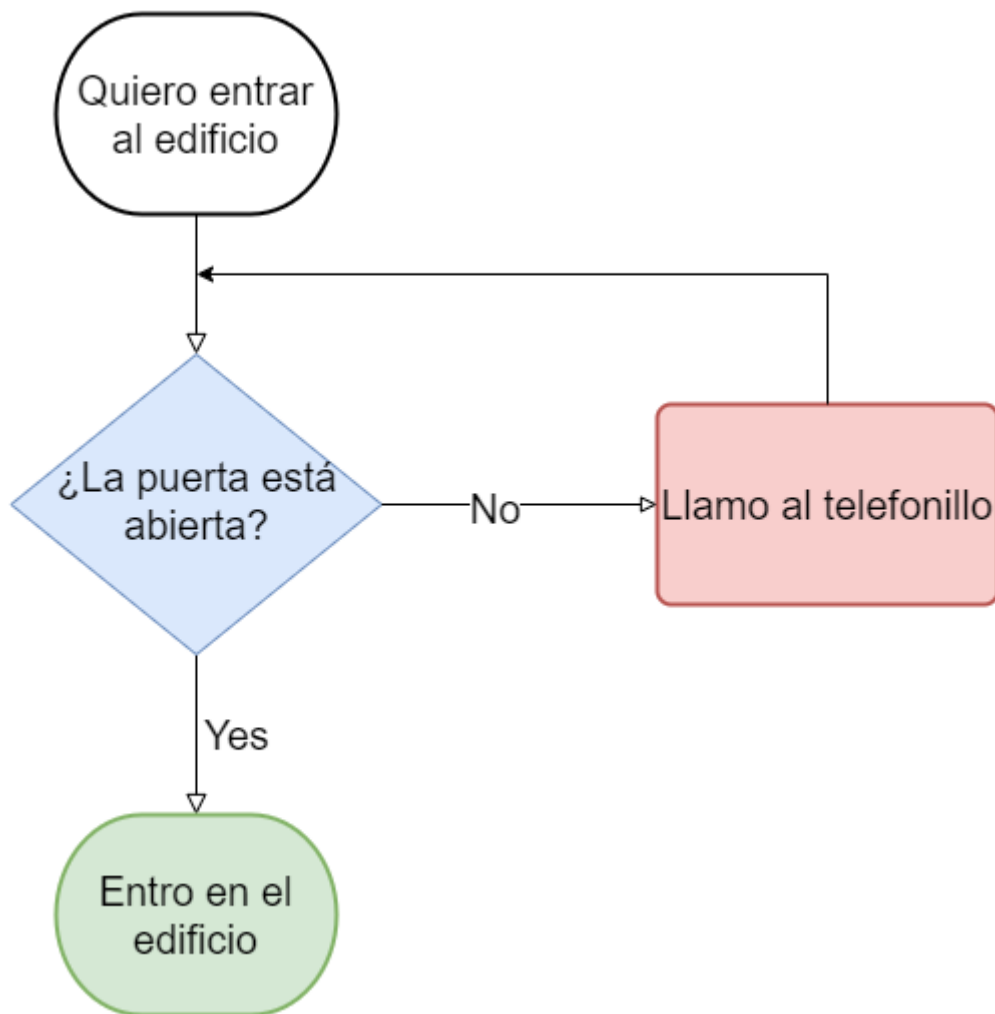
Dentro de este tipo de sentencias Java disponemos de tres instrucciones:

- While
- Do ... While
- For

### 2.1. While

La instrucción while consiste en realizar la comprobación de una condición y en el caso de cumplirse se ejecuta el bloque de código que tiene en su interior. Una vez ejecutado el bloque vuelve a comprobarse la condición, de tal forma que si se vuelve a cumplir, vuelve a ejecutar el bloque de código para luego volver a comprobar la condición, y así hasta que no se cumpla dicha condición y saltaremos a la siguiente instrucción.

Debemos tener en cuenta que durante la primera comprobación de la condición esta no se cumple, nunca se ejecutará el bloque de instrucción dentro del while.



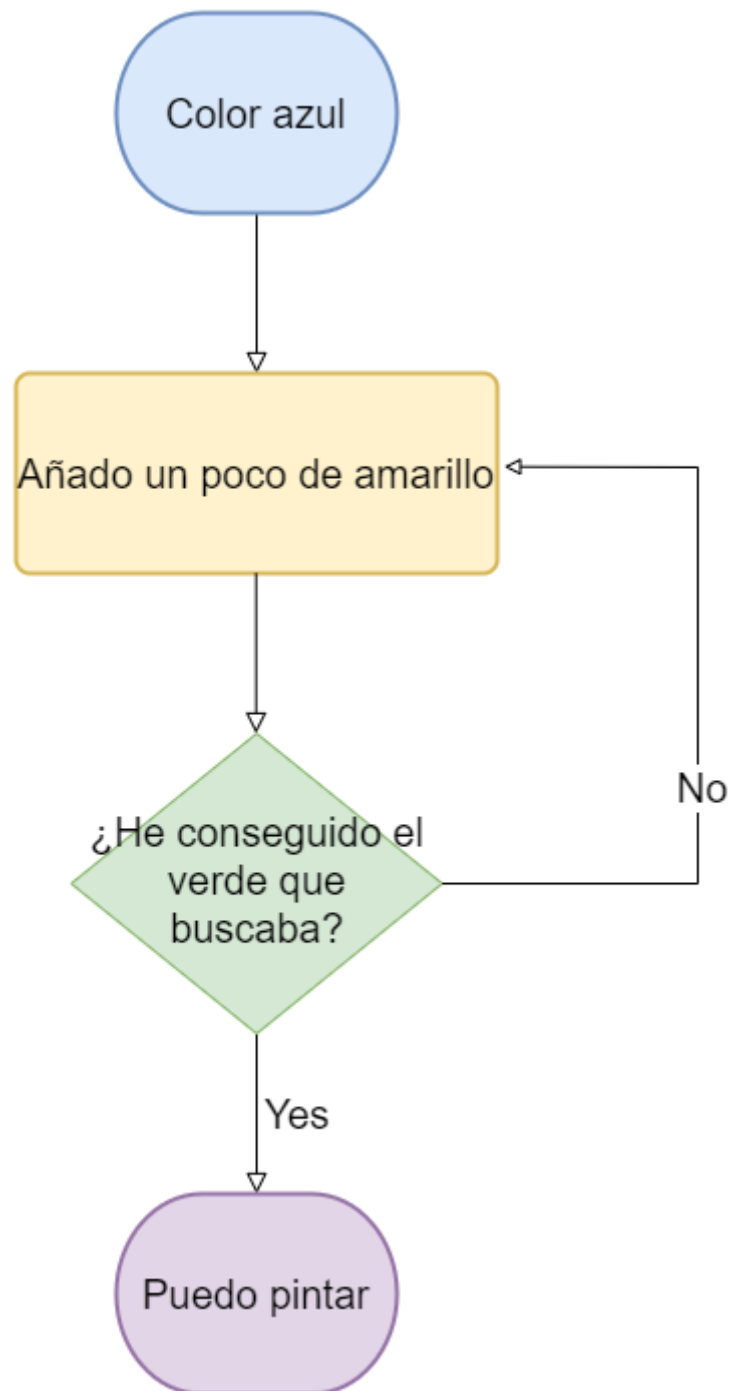
La estructura de instrucción while es la siguiente:

```
while (condición) {  
    bloque de instrucciones  
}
```

```
int num = 5;  
while (num < 10){  
    System.out.println(num);  
    num++;  
}
```

## 2.2. Do ... While

La instrucción `do ... while` es similar a la instrucción `while`, con la única diferencia de que la comprobación de la condición se realiza al finalizar la ejecución del bloque de código que contiene. De esta forma siempre se ejecutará al menos una vez el bloque de instrucción dentro del `do ... while`.



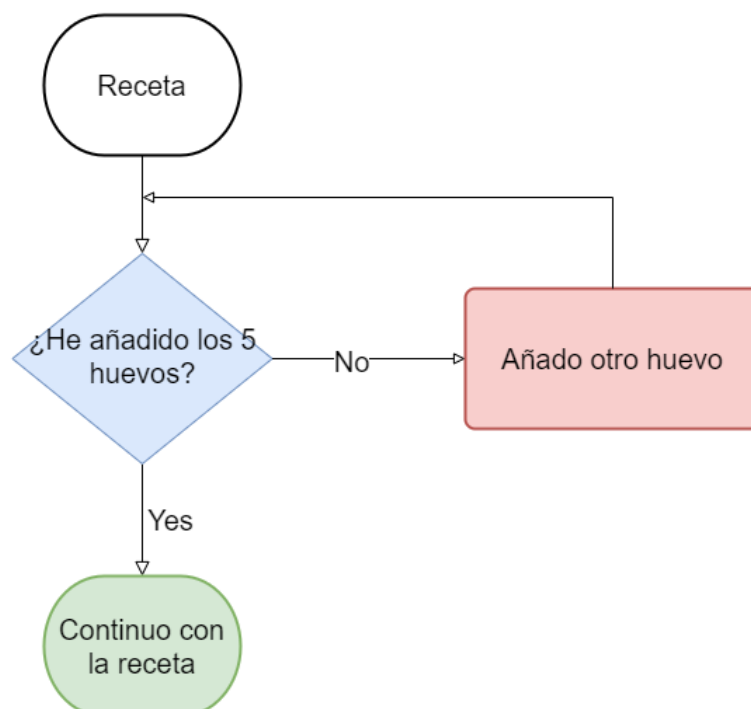
La instrucción `do ... while` se estructura de la siguiente manera:

```
do {  
    bloque de código  
} while (condición);
```

```
num = 100;  
do {  
    System.out.println(num);  
    num--;  
} while (num >= 0);
```

### 2.3. For

La instrucción `for` es utilizada, normalmente, cuando conocemos las veces que se va a ejecutar el bucle. Esta instrucción es particular puesto que debe de haber una variable que inicializaremos y que utilizaremos como contador. Al igual que la instrucción `while`, la comprobación de la condición se realiza previamente a la ejecución del bloque de instrucciones que contiene y, por tanto, puede ocurrir que dicho bloque no se ejecute nunca.



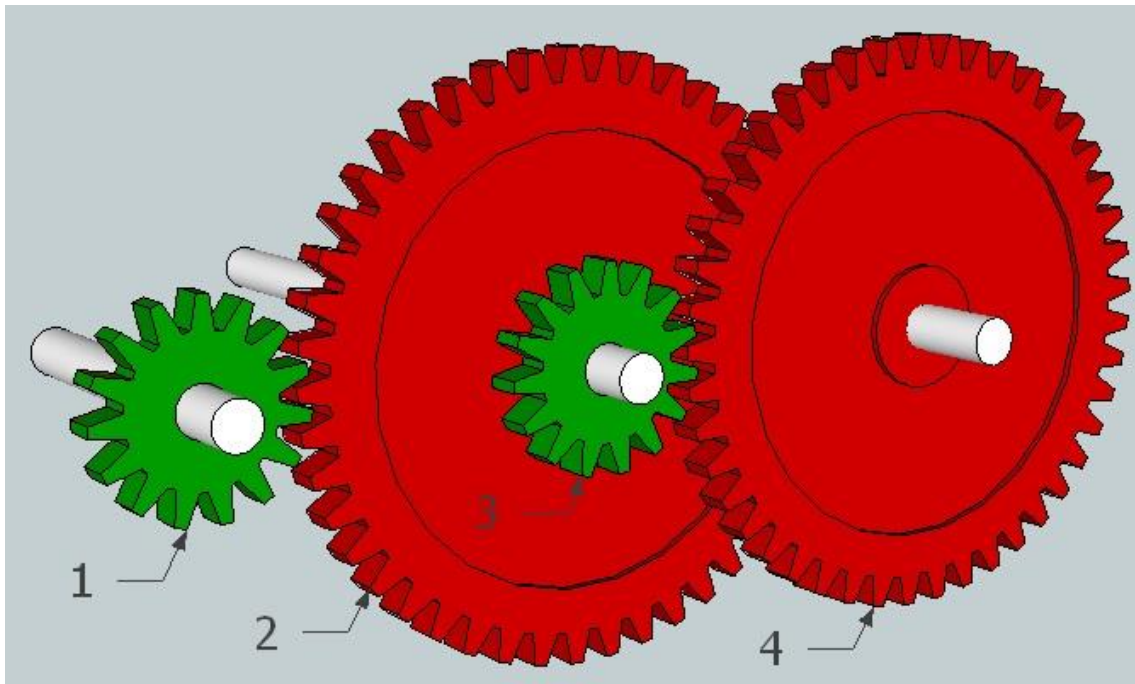
La instrucción for tiene la siguiente estructura:

```
for (int contador = valorInicial; condición; modificaciónContador){  
    bloque de código  
}
```

```
for (int i = 1; i <= 10; i++){  
    System.out.println("Escrito " + i + " veces");  
}
```

## 2.4. Bucles anidados

Al igual que las sentencias condicionales también podemos anidar bucles, del mismo tipo o de distinto tipo.



Estos bucles pueden ser de dos tipos:



### 2.4.1. Bucles independientes

En esta modalidad los bucles no están relacionados entre sí, es decir, que el funcionamiento de uno de afecte al otro.



```
int contWhile = 0;
int contDoWhile = 5;
while (contWhile <= 50){
    do{
        System.out.println(contDoWhile + " ");
        contDoWhile--;
    } while (contDoWhile >= 0);
    System.out.println();
    contWhile++;
}
```

### 2.4.2. Bucles dependientes

En esta modalidad los bucles están relacionados, de tal forma que el funcionamiento de uno afecta al otro.

| TABLA MULTIPLICAR |             |             |             |               |
|-------------------|-------------|-------------|-------------|---------------|
| 1 x 1 = 1         | 2 x 1 = 2   | 3 x 1 = 3   | 4 x 1 = 4   | 5 x 1 = 5     |
| 1 x 2 = 2         | 2 x 2 = 4   | 3 x 2 = 6   | 4 x 2 = 8   | 5 x 2 = 10    |
| 1 x 3 = 3         | 2 x 3 = 6   | 3 x 3 = 9   | 4 x 3 = 12  | 5 x 3 = 15    |
| 1 x 4 = 4         | 2 x 4 = 8   | 3 x 4 = 12  | 4 x 4 = 16  | 5 x 4 = 20    |
| 1 x 5 = 5         | 2 x 5 = 10  | 3 x 5 = 15  | 4 x 5 = 20  | 5 x 5 = 25    |
| 1 x 6 = 6         | 2 x 6 = 12  | 3 x 6 = 18  | 4 x 6 = 24  | 5 x 6 = 30    |
| 1 x 7 = 7         | 2 x 7 = 14  | 3 x 7 = 21  | 4 x 7 = 28  | 5 x 7 = 35    |
| 1 x 8 = 8         | 2 x 8 = 16  | 3 x 8 = 24  | 4 x 8 = 32  | 5 x 8 = 40    |
| 1 x 9 = 9         | 2 x 9 = 18  | 3 x 9 = 27  | 4 x 9 = 36  | 5 x 9 = 45    |
| 1 x 10 = 10       | 2 x 10 = 20 | 3 x 10 = 30 | 4 x 10 = 40 | 5 x 10 = 50   |
| 6 x 1 = 6         | 7 x 1 = 7   | 8 x 1 = 8   | 9 x 1 = 9   | 10 x 1 = 10   |
| 6 x 2 = 12        | 7 x 2 = 14  | 8 x 2 = 16  | 9 x 2 = 18  | 10 x 2 = 20   |
| 6 x 3 = 18        | 7 x 3 = 21  | 8 x 3 = 24  | 9 x 3 = 27  | 10 x 3 = 30   |
| 6 x 4 = 24        | 7 x 4 = 28  | 8 x 4 = 32  | 9 x 4 = 36  | 10 x 4 = 40   |
| 6 x 5 = 30        | 7 x 5 = 35  | 8 x 5 = 40  | 9 x 5 = 45  | 10 x 5 = 50   |
| 6 x 6 = 36        | 7 x 6 = 42  | 8 x 6 = 48  | 9 x 6 = 54  | 10 x 6 = 60   |
| 6 x 7 = 42        | 7 x 7 = 49  | 8 x 7 = 56  | 9 x 7 = 63  | 10 x 7 = 70   |
| 6 x 8 = 48        | 7 x 8 = 56  | 8 x 8 = 64  | 9 x 8 = 72  | 10 x 8 = 80   |
| 6 x 9 = 54        | 7 x 9 = 63  | 8 x 9 = 72  | 9 x 9 = 81  | 10 x 9 = 90   |
| 6 x 10 = 60       | 7 x 10 = 70 | 8 x 10 = 80 | 9 x 10 = 90 | 10 x 10 = 100 |

```

for(int i = 10; i >= 0 ; i--){
    for (int j = i; i <= 10; j++){
        System.out.print(j + " ");
    }
    System.out.println();
}

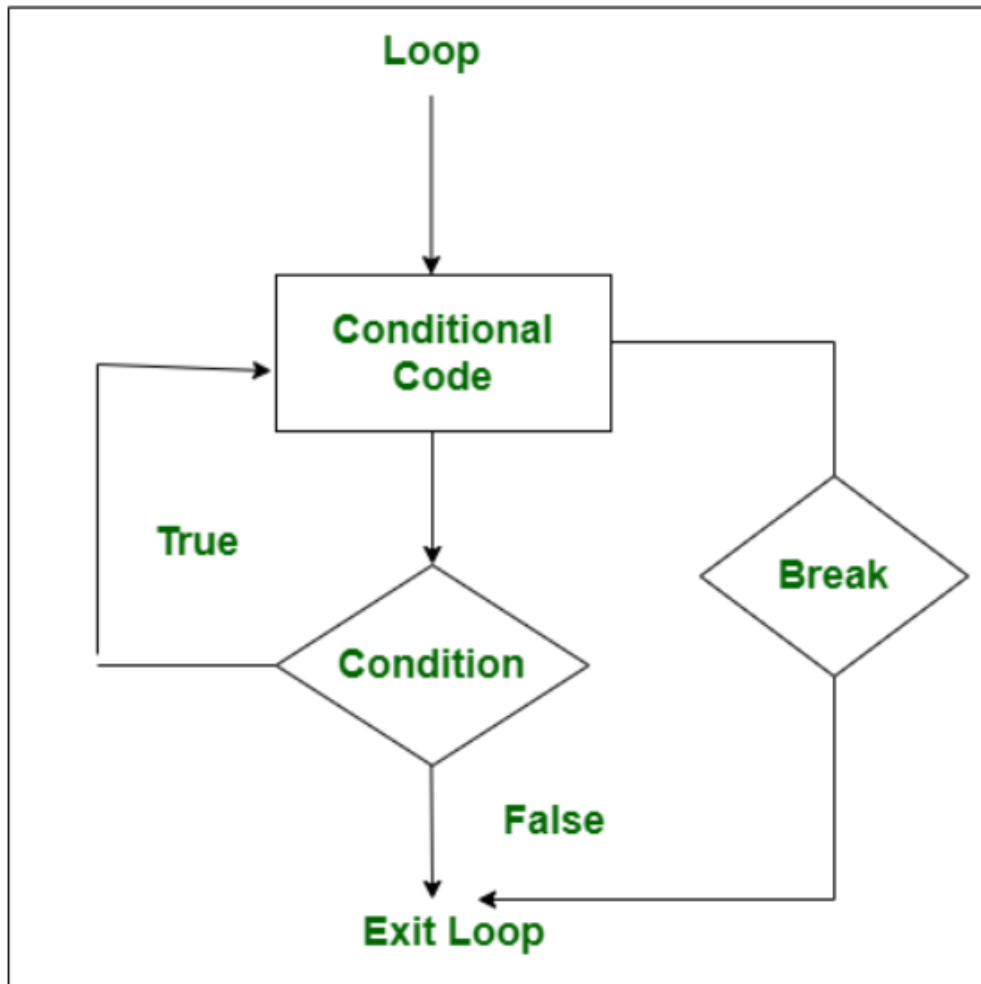
```

## 4. SENTENCIAS DE SALTO

Las sentencias de salto nos serán muy útiles cuando necesitemos interrumpir la ejecución de un bucle o saltar parte de su código para volver a comprobar la condición. En java disponemos de las instrucciones break y continue. Estas dos instrucciones podremos utilizarlas en cualquiera de los tres tipos de sentencias iterativas que hemos visto anteriormente.

### 4.1. Break

Con esta instrucción finalizaremos inmediatamente el bucle y continuaremos en la línea siguiente a la que finaliza este.

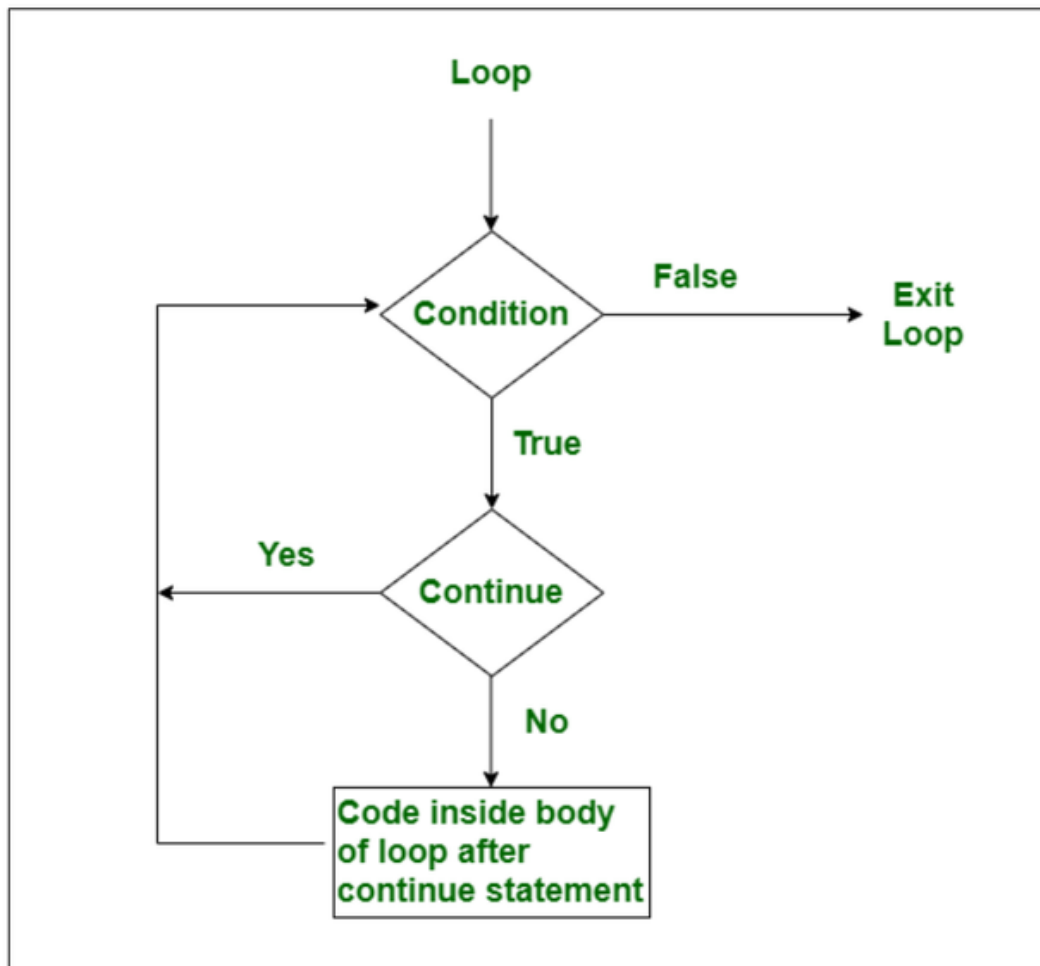


La forma de utilizarlo es simplemente escribiendo la instrucción “break;”.

```
num = 5;
while (num < 10){
    System.out.println(num);
    if (num == 8) {
        break;
    }
    num++;
}
System.out.println("He llegado hasta el número " + num);
```

## 4.2. Continue

Con la instrucción continue saltaremos el resto del contenido del bloque de instrucciones dentro del bucle y volveremos a comprobar la condición. En el caso de la instrucción for también realizará la modificación del contador correspondiente a la iteración.



La forma de utilizarlo es simplemente escribiendo la instrucción “continue;”.

```
for(int i=0; i<=10; i++){  
    if (i%2 == 0){  
        continue;  
    }  
    System.out.println("Voy por el " + i);  
}
```

## 5. PRUEBAS, DEPURACIÓN Y DOCUMENTACIÓN DE LA APLICACIÓN

### 5.1. Pruebas de programas

Cuando se habla de pruebas de software o de programa es común utilizar los términos “prueba” y “caso de prueba”. Su definición es la siguiente:

- **Prueba:** Es el proceso de ejecución de un programa con el intento deliberado de encontrar errores.
- **Caso de prueba:** Conjunto de entradas, condiciones de ejecución y resultados esperados diseñados para un objetivo particular de condición de prueba.

### 5.2. Errores de los programas

Durante el proceso de desarrollo de software, se pueden producir dos tipos de errores:

- **Errores de compilación:** se corresponde con el incumplimiento de reglas sintácticas del lenguaje de programación utilizado. Ocasiona que el programa no puede ejecutarse hasta que el programador no corrija ese error. Ejemplos de errores de compilación:
  - Palabras reservadas mal escritas.
  - Expresiones erróneas o incompletas.
  - Variables no declaradas.

Son detectados por el propio entorno de desarrollo en tiempo real durante su escritura o durante la compilación del código.

- **Errores lógicos o de ejecución:** comúnmente llamados bugs. Estos no evitan que el programa se pueda compilar con éxito, ya que no hay errores sintácticos y permiten su ejecución. Sin embargo, pueden provocar que el programa devuelva resultados erróneos, que no sean los esperados o pueden provocar que el programa termine antes de tiempo o no termine nunca. Ejemplos de errores lógicos o de ejecución son:

- Dividir un número por cero.
- Exceder un rango de valores posible en una variable o una estructura de datos.
- Crear un bucle infinito.

Para la localización y corrección de estos errores se usa una herramienta de gran utilidad, como es el depurador.

### 5.3. El depurador de código

El depurador permite supervisar la ejecución de los programas, para localizar y eliminar los errores lógicos. Un programa debe compilarse con éxito para posteriormente poder utilizarlo en el depurador. El depurador nos permite analizar todo el programa, mientras éste se ejecuta. Permite suspender la ejecución de un programa, examinar y establecer los valores de las variables, comprobar los valores devueltos por un determinado método, el resultado de una comparación lógica o relacional, etc.

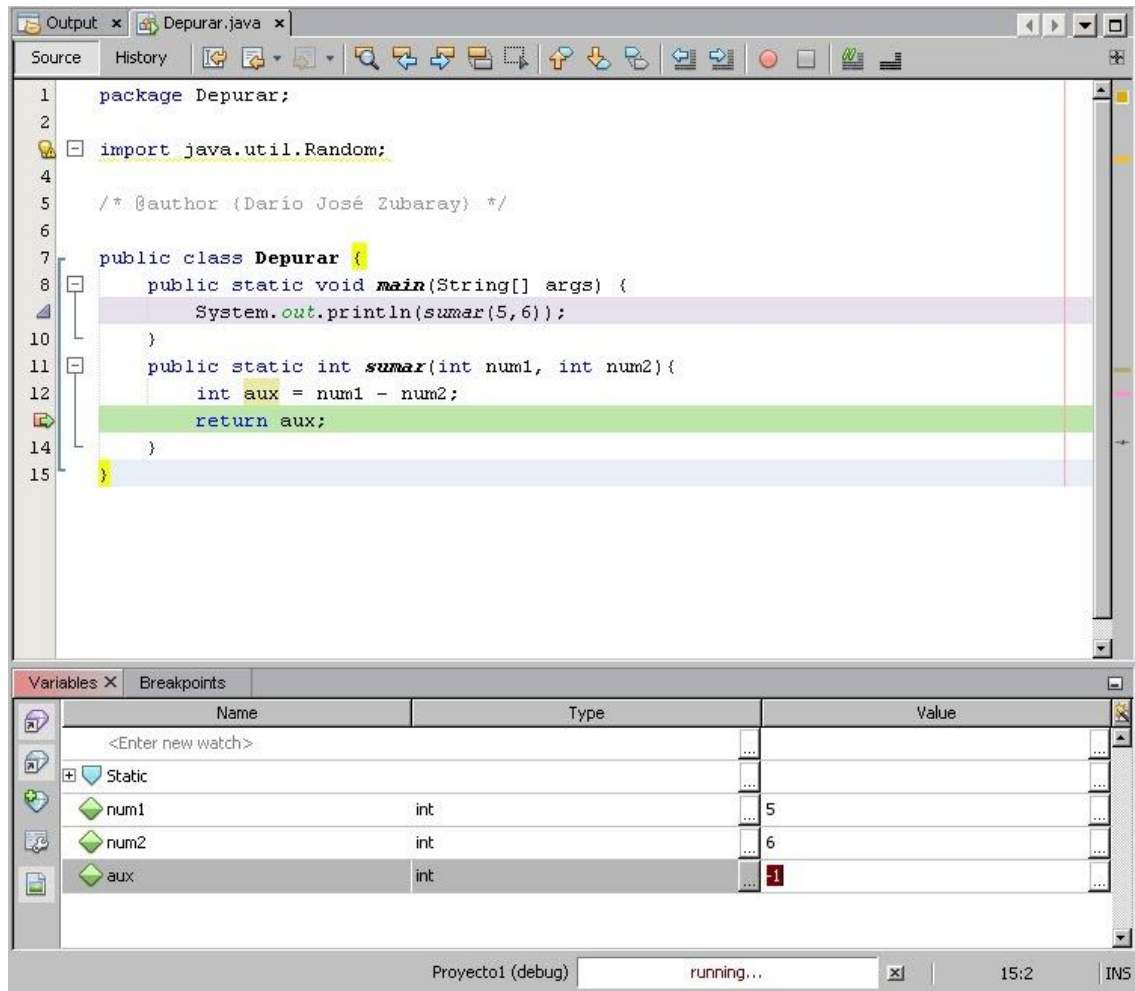
De este modo, el depurador permite analizar el flujo de ejecución del código y el estado de los datos conforme van siendo manipulados por el programa.

Para poder depurar un programa, podemos ejecutar el programa de diferentes formas, de manera que en función del problema que se quiera solucionar, resulte más sencillo un método u otro.

Existen los siguientes tipos de ejecución:

- **Paso a paso por instrucción:** algunas veces es necesario ejecutar un programa línea por línea para buscar y corregir errores lógicos.
- **Paso a paso por procedimiento:** permite introducir los parámetros que queremos a un método o función de nuestro programa, pero en vez de ejecutar instrucción por instrucción ese método, nos devuelve su resultado. Es útil, cuando hemos comprobado que un procedimiento funciona correctamente, y no nos interesa volver a depurarlo, solo nos interesa el valor que devuelve.

- **Ejecución hasta una instrucción:** el depurador ejecuta el programa y se detiene en la instrucción donde se encuentra el cursor, a partir de ese punto podemos hacer una depuración paso a paso o por procedimiento.
- **Ejecutar hasta el final del programa:** se ejecuta las instrucciones de un programa hasta el final, sin detenerse en las instrucciones intermedias.



## 5.4. Documentación

Para documentar nuestros programas lo realizaremos mediante comentarios, siguiendo las mismas indicaciones que dimos en su momento en la unidad de trabajo número 2.